

Évaluation Machine — Programmation Orientée Objet (C#)

Sujet : Gestion d'une Réserve Naturelle de Renards

Durée : 1h00 — Format : Sur Machine

Contexte :

Vous développez une application de gestion pour une réserve naturelle spécialisée dans le sauvetage, le soin et l'hébergement de renards roux. Votre mission est de modéliser les renards, les enclos et la réserve elle-même pour optimiser les placements et gérer l'alimentation.

Consignes Générales :

- Le code doit être écrit en C# valide et compiler sans erreur.
 - Respecter strictement l'encapsulation (attributs privés, propriétés publiques).
 - Respecter les conventions de nommage (PascalCase pour les classes/méthodes/propriétés, camelCase pour les variables locales).
-

Partie 1 : Le Renard (6 points)

La classe `Renard` représente un animal hébergé dans la réserve.

Travail à faire :

Créer la classe `Renard` avec les éléments suivants :

1. Propriétés :

- `Id` (string) : Identifiant unique (ex: "REN-001").
- `Surnom` (string) : Nom donné au renard (ex: "Rox").
- `Age` (int) : Âge du renard en mois.
- `Poids` (double) : Poids du renard en kilogrammes.
- `NiveauSociabilite` (int) : Valeur de 1 à 10 (1 = très craintif/solitaire, 10 = très joueur/sociable).
- `EstMalade` (bool) : État de santé de l'animal.

2. Constructeur :

- Initialiser l'`Id`, le `Surnom`, l'`Age`, le `Poids` et le `NiveauSociabilite`.
- Par défaut, un renard arrive en bonne santé (`EstMalade` = `false`).

3. Méthode `CalculerIndiceCompatibilite(Renard autreRenard)` :

- Retourner un `int` représentant le pourcentage de chances que deux renards s'entendent bien (de 0 à 100).
- **Algorithme de calcul du score de base (démarré à 50) :**
 - Si la différence d'âge entre les deux renards est **strictement inférieure à 6 mois**, ajouter **20 points** (ils ont le même âge, ils jouent ensemble).
 - Si la différence de poids est **supérieure à 3 kg**, retirer **15 points** (risque de domination).
 - Si l'un des deux renards (ou les deux) `EstMalade`, retirer **30 points** (l'animal malade a besoin de calme).
 - Ajouter à ce score la moyenne de leurs deux `NiveauSociabilite` multipliée par 2

(ex: si l'un a 4 et l'autre 8, la moyenne est 6, on ajoute 12 points).

- **Contrainte** : Le résultat retourné doit impérativement être compris entre 0 et 100 (utiliser des conditions ou `Math.Clamp` si vous connaissez).

Partie 2 : L'Enclos (6 points)

La réserve est divisée en plusieurs enclos pouvant accueillir les renards.

Travail à faire :

Créer la classe `Enclos` :

1. **Propriétés & Attributs** :
 - `Nom` (string) : Nom de l'enclos (ex: "La Grande Clairière").
 - `CapaciteMax` (int) : Nombre maximum de renards.
 - `_pensionnaires` : Une liste privée (`List<Renard>`) qui stocke les renards présents.
2. **Constructeur** :
 - Initialiser le `Nom` et la `CapaciteMax`. Instancier la liste `_pensionnaires`.
3. **Méthode `PeutAccueillir()`** :
 - Retourner `true` si le nombre actuel de renards est strictement inférieur à la `CapaciteMax`.
Retourner `false` sinon.
4. **Méthode `AjouterRenard(Renard r)`** :
 - Vérifier via `PeutAccueillir()` s'il reste de la place.
 - Si oui : ajouter le renard à la liste et retourner `true`.
 - Si non : retourner `false`.
5. **Méthode `VerifierHarmonie()`** :
 - Retourner un `bool`.
 - Cette méthode vérifie si l'enclos est pacifique. S'il y a moins de 2 renards, retourner `true`.
 - Sinon, comparer **le premier renard de la liste avec le deuxième**. S'ils ont un indice de compatibilité (via la méthode de la Partie 1) **strictement inférieur à 40**, retourner `false` (alerte conflit). Sinon, `true`. (Pour simplifier, on ne compare que les 2 premiers).

Partie 3 : La Réserve (8 points)

La réserve gère l'ensemble des enclos et l'arrivée des nouveaux pensionnaires.

Travail à faire :

Créer la classe `ReserveNaturelle` :

1. **Attributs** :
 - `NomReserve` (string).
 - `_secteurs` : Une liste privée (`List<Enclos>`).
2. **Constructeur** :
 - Initialiser le nom de la réserve et instancier la liste d'enclos.
3. **Méthode `AjouterEnclos(Enclos e)`** :
 - Ajouter un enclos à la liste `_secteurs`.

4. Méthode `PlacerNouveauRenard(Renard nouveauRenard)` :

- **Algorithme de placement :**
 - a. Parcourir tous les enclos de la réserve.
 - b. Trouver le **premier** enclos qui `PeutAccueillir()` le renard.
 - c. Utiliser `AjouterRenard` de cet enclos.
 - d. Afficher dans la console : *"Le renard [Surnom] a été placé dans l'enclos [Nom de l'enclos]"*.
 - e. Retourner `true` pour stopper la recherche.
- Si aucun enclos n'a de place, afficher *"Urgence : Plus de place dans la réserve pour [Surnom] !"* et retourner `false` .

5. Simulation dans le `Main` :

- Créer une réserve "Le Terrier Sûr".
- Ajouter 2 enclos : "Bois" (capacité 2) et "Colline" (capacité 5).
- Créer 3 renards aux caractéristiques différentes (faire en sorte que deux d'entre eux aient une forte différence de poids ou soient malades pour faire chuter leur compatibilité).
- Utiliser `PlacerNouveauRenard` sur votre réserve pour y insérer vos 3 renards un par un.
- Appeler la méthode `VerifierHarmonie()` sur l'enclos "Bois" et afficher le résultat dans la console (ex: "L'enclos Bois est-il harmonieux ? True/False").